

Preface

This project, **Teach for Air Quality Monitoring**, aims to provide clear knowledge, practical skills, and hands-on experience related to measuring, analyzing, and interpreting air quality data.

Air quality directly impacts our health, environment, and quality of life, making it crucial to monitor and understand the air we breathe.

The developed Air Quality Monitoring Dashboard empowers users by providing real-time, location-based environmental information using modern web technologies such as HTML, CSS, JavaScript, Python, and Flask.

The dashboard retrieves information from external Air Quality APIs and dynamically presents pollutant values including PM2.5, PM10, Carbon Dioxide (CO₂), and Air Quality Index (AQI) through a clean and interactive interface.

This project provides students, researchers, and environmental enthusiasts with practical exposure to API integration, web application development, and environmental data visualization.

Contents

Preface	1
Abstract	6
1 Introduction	7
1.1 Introduction	7
1.2 Features of the Proposed Dashboard	8
1.3 Advantages	8
2 Problem Statement	9
2.1 Problem Statement	9
2.2 Existing System	9
2.2.1 Advantages	10
2.2.2 Disadvantages	10
2.3 Proposed System	11
3 Goals and Objectives	12
3.1 Objective	12
3.2 Project Goals	12
3.3 Project Objectives	13
3.4 Expected Outcomes	13
4 Scope and Limitations	14
4.1 Project Scope	14
4.2 Project Limitations	15
5 System Analysis	16
5.1 Existing System Analysis	16
5.1.1 Advantages	16
5.1.2 Disadvantages	16
5.2 Proposed System Analysis	17
5.3 Working Procedure	17
5.4 Feasibility Study	17

5.5	System Requirement Analysis	18
5.5.1	Functional Requirements	18
5.5.2	Non-Functional Requirements	19
6	System Design	20
6.1	Entity Relationship Diagram	20
6.2	Activity Diagram	21
6.3	Use Case Diagram	22
6.4	User Module	23
7	Implementation Details	24
7.1	Software Requirements	24
7.2	Hardware Requirements	24
7.3	Development Tools	25
7.4	Implementation Workflow	25
8	Output	26
8.1	Home Page	26
8.2	Dashboard	27
8.3	Charts	27
9	Future Scope	28
10	Conclusion	29

List of Figures

6.1	Entity Relationship Diagram	20
6.2	Activity Diagram	21
6.3	Use Case Diagram	22
8.1	Home Page	26
8.2	Air Quality Dashboard	27
8.3	Air Quality Charts	27

List of Tables

5.1	Feasibility Study	18
7.1	Software Requirements	24
7.2	Hardware Requirements	24

Abstract

Air pollution has become one of the most significant environmental challenges affecting public health, ecosystems, and climate worldwide. Timely access to accurate air quality information enables individuals to make informed decisions regarding outdoor activities and health precautions.

The Teach for Air Quality Monitoring project presents a real-time web-based dashboard that automatically determines the user's geographical location using the browser's Geolocation API. The coordinates are transmitted to a Flask-based backend, which securely communicates with the World Air Quality Index (WAQI) API to retrieve current environmental data.

The retrieved information includes Air Quality Index (AQI), PM2.5, PM10, Carbon Dioxide (CO₂), temperature, humidity, and other environmental parameters. The data is processed and displayed through a responsive HTML, CSS, and JavaScript interface featuring dynamic updates and intuitive visualization.

The proposed system eliminates the limitations of static monitoring systems by providing location-aware, real-time environmental information without requiring dedicated hardware installations. The system is scalable, user-friendly, and can be extended in the future through IoT-based sensors, historical data analysis, predictive analytics, and smart city integration.

1 Introduction

1.1 Introduction

Air quality has become one of the most important environmental concerns in recent years. Rapid urbanization, increasing industrial activities, and the growing number of vehicles have significantly increased the concentration of harmful pollutants in the atmosphere. These pollutants directly affect human health, environmental sustainability, and climate change.

To address this issue, the **Teach for Air Quality Monitoring** project presents a real-time web-based dashboard capable of displaying location-specific air quality information. The application is developed using modern web technologies including HTML, CSS, JavaScript, Python, and Flask.

Unlike conventional static monitoring systems, the proposed dashboard automatically detects the user's geographical location using the browser's Geolocation API. The obtained latitude and longitude are securely transmitted to a Flask backend server, which communicates with the World Air Quality Index (WAQI) API to retrieve real-time environmental data.

The dashboard displays important air-quality parameters such as:

- Air Quality Index (AQI)
- PM_{2.5} concentration
- PM₁₀ concentration
- Carbon Dioxide (CO₂)
- Temperature
- Humidity

The retrieved data are presented through a clean, responsive, and user-friendly interface with dynamic updates and intuitive visualization techniques.

The system enables users to monitor pollution levels instantly without requiring dedicated hardware installations. By providing reliable environmental information,

the dashboard assists users in making informed decisions regarding outdoor activities and health precautions.

Furthermore, the proposed system serves as an educational platform for students interested in environmental monitoring, API integration, full-stack web development, and real-time data visualization.

1.2 Features of the Proposed Dashboard

The major features of the proposed dashboard include:

- Automatic location detection using Geolocation API
- Real-time Air Quality Index (AQI)
- Pollutant concentration monitoring
- Flask-based backend API integration
- Responsive web interface
- Dynamic data updates
- Health-oriented environmental awareness
- Future support for IoT integration

1.3 Advantages

The proposed system provides several advantages over traditional monitoring systems.

1. Real-time environmental monitoring
2. Automatic location detection
3. Simple and interactive interface
4. Accurate pollution information
5. Easy deployment
6. Low development cost
7. Highly scalable architecture
8. Platform-independent web application

2 Problem Statement

2.1 Problem Statement

Air pollution has become a major public health concern across the world. Although numerous online platforms provide air quality information, most of them suffer from one or more of the following limitations:

- Static information
- City-level pollution values only
- Lack of personalized monitoring
- No automatic location detection
- Poor visualization
- Separate mobile application requirements
- Delayed data updates

Consequently, users often struggle to obtain accurate pollution information corresponding to their current geographical location. This makes it difficult to take timely precautions against harmful environmental conditions.

The absence of a unified, lightweight, and location-aware web application creates the need for a system capable of retrieving and displaying hyper-local air quality information in real time.

The proposed Air Quality Monitoring Dashboard addresses these limitations by combining browser-based geolocation, Flask server-side processing, and external Air Quality APIs into a single integrated web application.

2.2 Existing System

The existing implementation consists primarily of a static web page developed using HTML, CSS, and JavaScript.

The displayed values are manually entered and do not represent actual environmental conditions.

Typical characteristics include:

- Static AQI values
- No backend processing
- No API communication
- No database
- No location awareness
- Manual updates
- Demonstration purposes only

2.2.1 Advantages

- Very simple implementation
- Fast loading
- Easy to understand
- No internet dependency

2.2.2 Disadvantages

- No real-time monitoring
- Inaccurate information
- No personalized data
- No API integration
- Cannot support practical applications
- Not suitable for public deployment

2.3 Proposed System

The proposed system transforms the static webpage into a dynamic client-server application.

The overall workflow is summarized below.

1. User opens the dashboard.
2. Browser requests location permission.
3. Latitude and longitude are obtained.
4. Coordinates are sent to Flask backend.
5. Flask communicates with WAQI API.
6. Environmental data are retrieved.
7. JSON response is processed.
8. Dashboard updates automatically.

The proposed system provides reliable, location-aware environmental monitoring through an intuitive user interface.

3 Goals and Objectives

3.1 Objective

The primary objective of this project is to design and develop a responsive web application capable of providing users with accurate, real-time, and location-based air quality information.

The dashboard aims to simplify environmental monitoring by integrating browser geolocation services with external Air Quality APIs through a Flask backend server.

3.2 Project Goals

The major goals of the project are listed below.

1. Automatically obtain the user's geographic location.
2. Integrate Flask with external WAQI APIs.
3. Display Air Quality Index (AQI).
4. Display $\text{PM}_{2.5}$ concentration.
5. Display PM_{10} concentration.
6. Display Carbon Dioxide (CO_2) values.
7. Display temperature and humidity.
8. Provide responsive web design.
9. Support manual data refresh.
10. Improve environmental awareness.
11. Deliver fast response time.
12. Maintain a clean and user-friendly interface.

13. Allow future IoT integration.
14. Support historical data expansion.
15. Provide scalable software architecture.

3.3 Project Objectives

The specific objectives of the proposed system include:

- Develop a Flask-based backend.
- Build a responsive HTML/CSS frontend.
- Implement JavaScript Geolocation API.
- Integrate external Air Quality APIs.
- Parse JSON responses efficiently.
- Display environmental parameters dynamically.
- Improve usability using responsive design.
- Ensure secure API communication.
- Handle API failures gracefully.
- Maintain modular software architecture.

3.4 Expected Outcomes

After successful implementation, the system will provide:

- Real-time AQI monitoring
- Automatic location detection
- Dynamic pollution visualization
- Fast API communication
- Improved user awareness
- Educational value for students
- Foundation for future smart-city applications

4 Scope and Limitations

4.1 Project Scope

The scope of the proposed Air Quality Monitoring Dashboard is focused on providing real-time, location-based environmental information through a web application.

The project mainly covers the following areas:

1. Development of a responsive web interface using HTML5, CSS3, and JavaScript.
2. Automatic retrieval of the user's geographical coordinates using the browser's Geolocation API.
3. Backend development using Python Flask.
4. Integration with external Air Quality APIs such as WAQI.
5. Retrieval and processing of JSON data.
6. Display of environmental parameters including AQI, PM_{2.5}, PM₁₀, CO₂, temperature, and humidity.
7. Dynamic updating of pollution information.
8. User-friendly interface with responsive design.
9. Easy deployment on local or cloud servers.

4.2 Project Limitations

Although the system provides accurate real-time monitoring, it has several limitations.

1. The application depends entirely on external Air Quality APIs.
2. Internet connectivity is mandatory.
3. The browser must support Geolocation API.
4. Users must grant location permission.
5. No database is currently used for historical storage.
6. Historical AQI trends are unavailable.
7. Offline functionality is not supported.
8. Limited pollutants are displayed according to API availability.
9. Accuracy depends upon the API provider.

5 System Analysis

5.1 Existing System Analysis

The existing Air Quality Monitoring webpage is primarily a static demonstration developed using HTML, CSS, and JavaScript.

Instead of retrieving live environmental information, the webpage displays manually entered values.

5.1.1 Advantages

- Lightweight implementation
- Easy to develop
- Fast loading
- No backend server required
- No API dependency

5.1.2 Disadvantages

- No real-time monitoring
- Static information
- No automatic location detection
- No API integration
- No backend processing
- Cannot provide personalized pollution data
- Unsuitable for real-world deployment

5.2 Proposed System Analysis

The proposed Air Quality Monitoring Dashboard converts the existing static webpage into a complete client-server web application.

The browser automatically retrieves the user's geographical location using the Geolocation API.

The coordinates are sent securely to the Flask backend.

The Flask server communicates with the World Air Quality Index (WAQI) API to retrieve real-time environmental data.

The backend processes the received JSON response and returns only the required parameters to the frontend.

The frontend dynamically updates all environmental information without requiring manual intervention.

5.3 Working Procedure

The overall working of the proposed system is shown below.

1. User opens the webpage.
2. Browser requests location permission.
3. Latitude and longitude are obtained.
4. Coordinates are sent to Flask backend.
5. Flask creates API request.
6. WAQI server processes request.
7. JSON response is received.
8. Flask extracts required information.
9. Data is sent to frontend.
10. Dashboard updates automatically.

5.4 Feasibility Study

The feasibility of the proposed system is analyzed under different aspects.

Table 5.1: Feasibility Study

Feasibility Type	Assessment	Remarks
Technical	Highly Feasible	Uses HTML, CSS, JavaScript, Python Flask, and WAQI API. No specialized hardware required.
Economic	Highly Feasible	Uses open-source software and free API services with minimal development cost.
Operational	Feasible	Simple interface with low maintenance requirements and easy usability.
Schedule	Highly Feasible	Project can be completed within the planned academic duration.
Legal	Feasible	Uses publicly available API services following provider policies.

5.5 System Requirement Analysis

5.5.1 Functional Requirements

The system shall:

- Detect the user's location.
- Retrieve latitude and longitude.
- Connect with Flask backend.
- Access WAQI API.
- Parse JSON responses.
- Display AQI.
- Display $PM_{2.5}$.
- Display PM_{10} .
- Display CO_2 concentration.
- Display temperature.
- Display humidity.
- Refresh environmental data.
- Handle API errors.

5.5.2 Non-Functional Requirements

- Fast response time
- Responsive design
- Easy usability
- High reliability
- Secure API communication
- Scalability
- Maintainability
- Cross-browser compatibility

6 System Design

6.1 Entity Relationship Diagram

The ER Diagram illustrates the logical relationship between users, location coordinates, environmental parameters, and API responses.

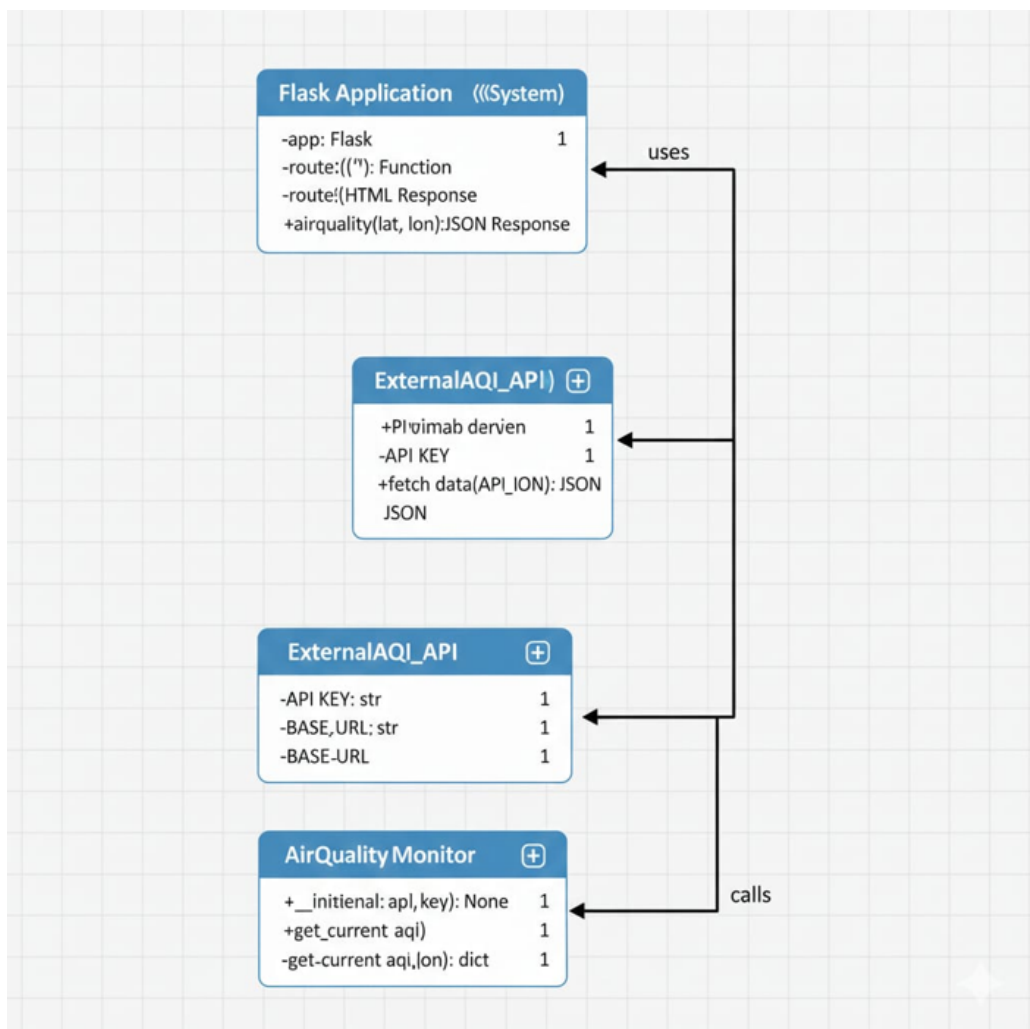


Figure 6.1: Entity Relationship Diagram

6.2 Activity Diagram

The Activity Diagram illustrates the workflow of retrieving and displaying air quality information.

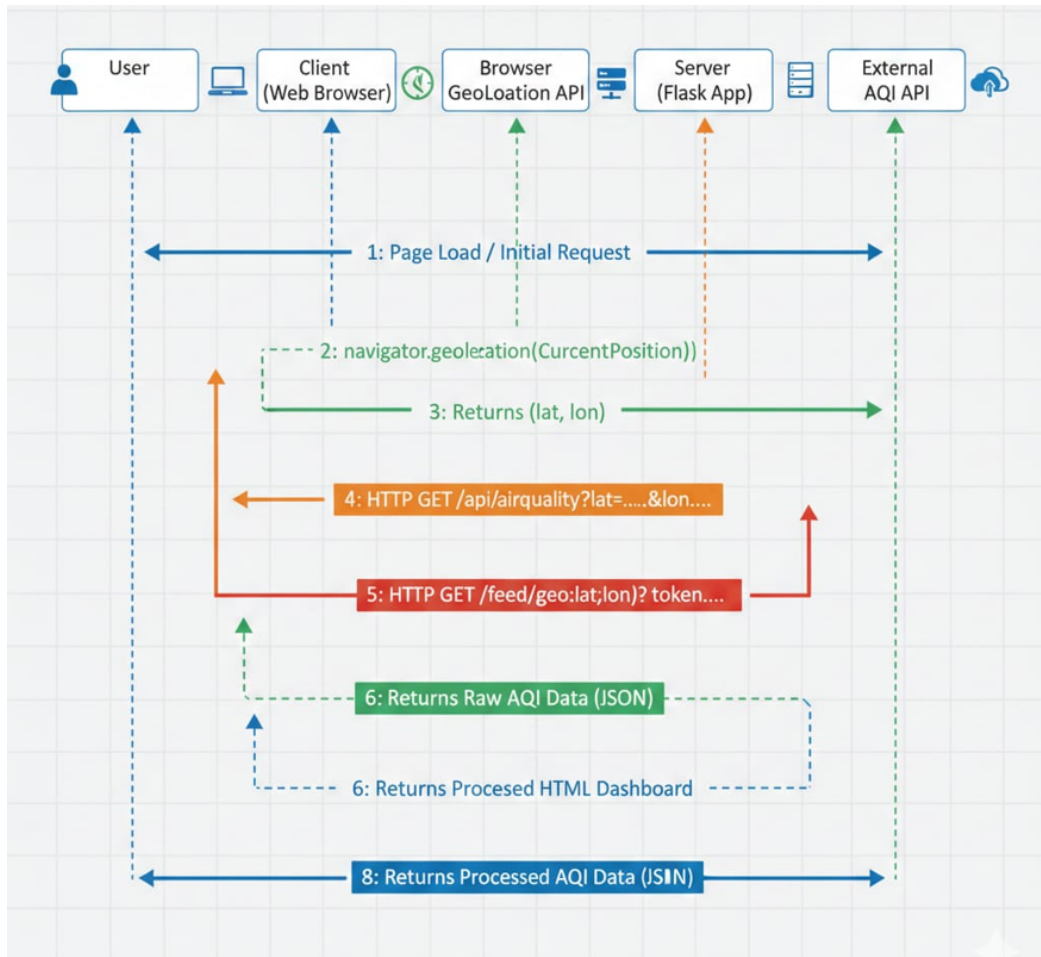


Figure 6.2: Activity Diagram

6.3 Use Case Diagram

The Use Case Diagram describes the interaction between the user and the Air Quality Monitoring Dashboard.

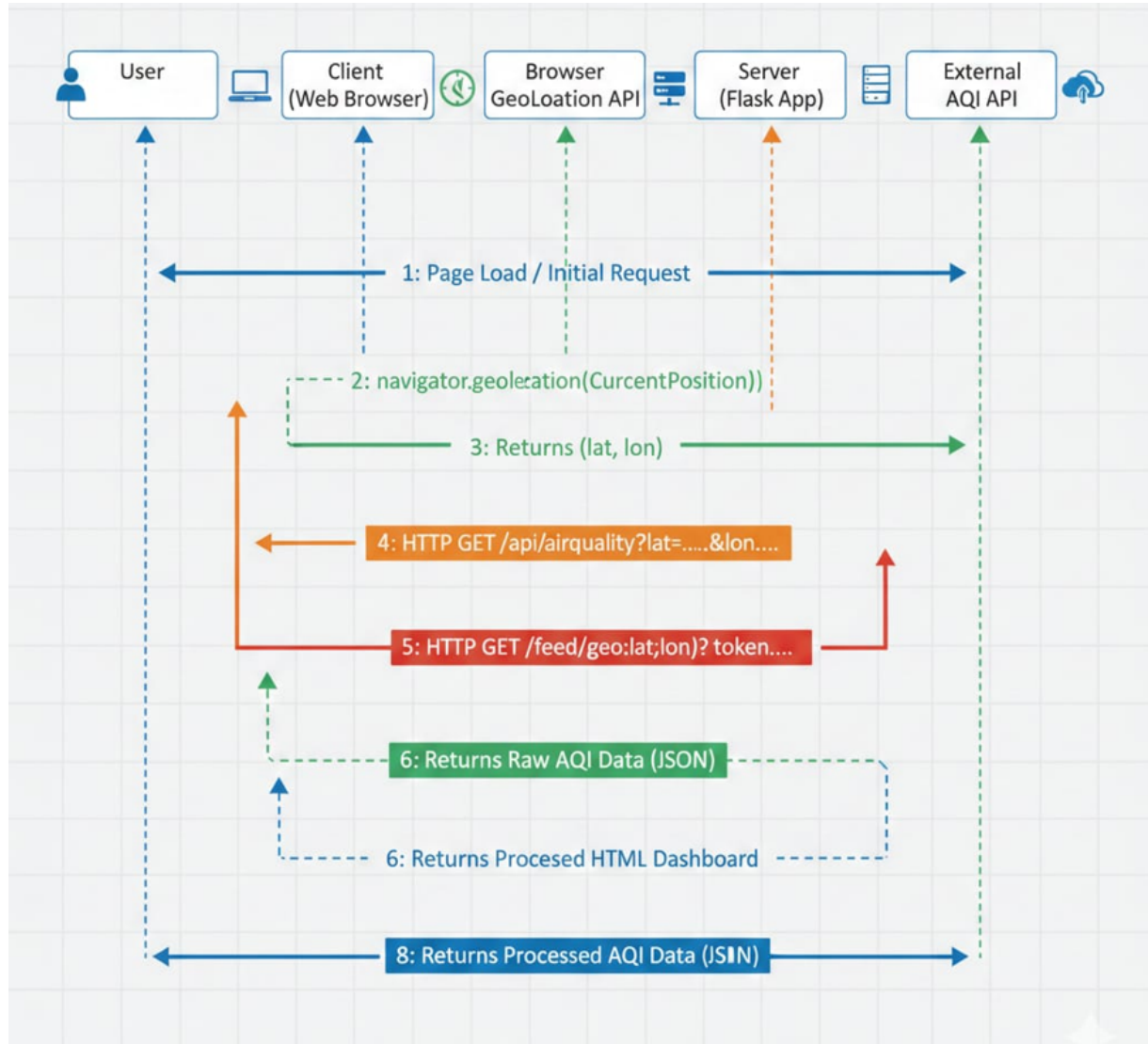


Figure 6.3: Use Case Diagram

6.4 User Module

The User Module is responsible for all user interactions within the application.

Major functionalities include:

- Opening the dashboard
- Granting location permission
- Refreshing environmental data
- Viewing AQI
- Viewing pollutant concentrations
- Monitoring weather information
- Understanding health recommendations

7 Implementation Details

7.1 Software Requirements

The software components used during the development of the proposed system are listed in Table 7.1.

Table 7.1: Software Requirements

Software	Description
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.x
Frontend	HTML5, CSS3, JavaScript
Backend Framework	Flask
Libraries	requests, Flask
Database	Not Used
Editor	Visual Studio Code / PyCharm
Browser	Google Chrome / Microsoft Edge
API	World Air Quality Index (WAQI) API
Version Control	Git and GitHub

7.2 Hardware Requirements

Table 7.2 presents the minimum hardware requirements.

Table 7.2: Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Storage	500 MB Free Space
Internet Connection	Required
GPS Enabled Device	Required for Geolocation
Display	1366 × 768 Resolution

7.3 Development Tools

The project was developed using the following technologies.

- HTML5
- CSS3
- JavaScript
- Python
- Flask Framework
- Requests Library
- WAQI API
- GitHub
- Visual Studio Code

7.4 Implementation Workflow

The implementation follows the client-server architecture.

1. User opens the dashboard.
2. Browser requests location permission.
3. Latitude and longitude are obtained.
4. Coordinates are transmitted to Flask.
5. Flask sends request to WAQI API.
6. API returns JSON data.
7. Required information is extracted.
8. Data is returned to the frontend.
9. Dashboard updates automatically.

8 Output

The following screenshots illustrate the working of the developed Air Quality Monitoring Dashboard.

8.1 Home Page

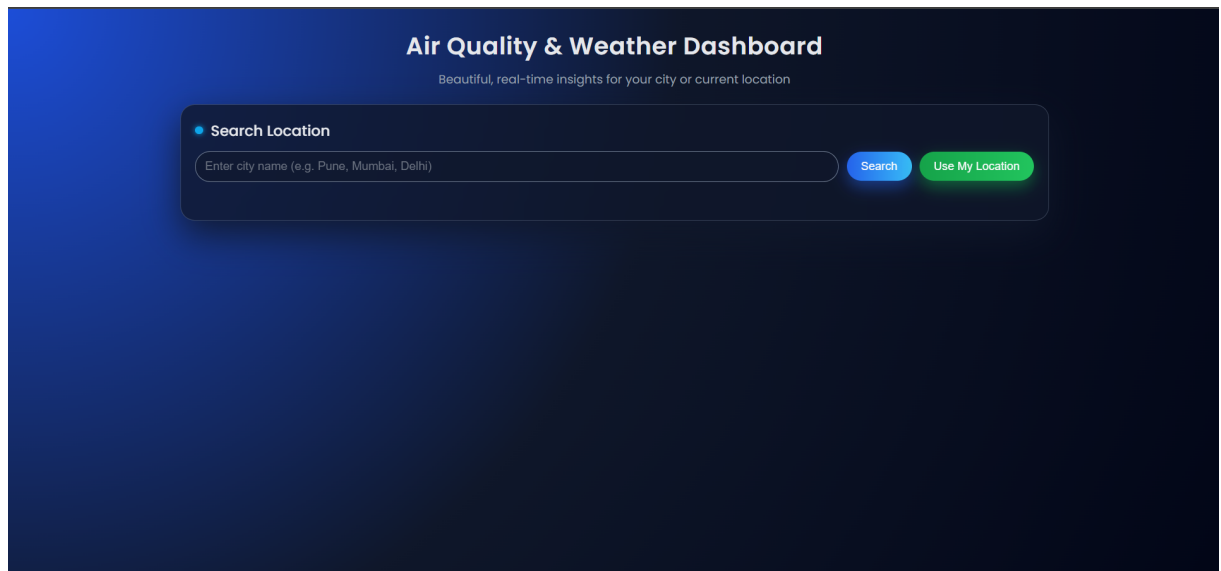


Figure 8.1: Home Page

8.2 Dashboard

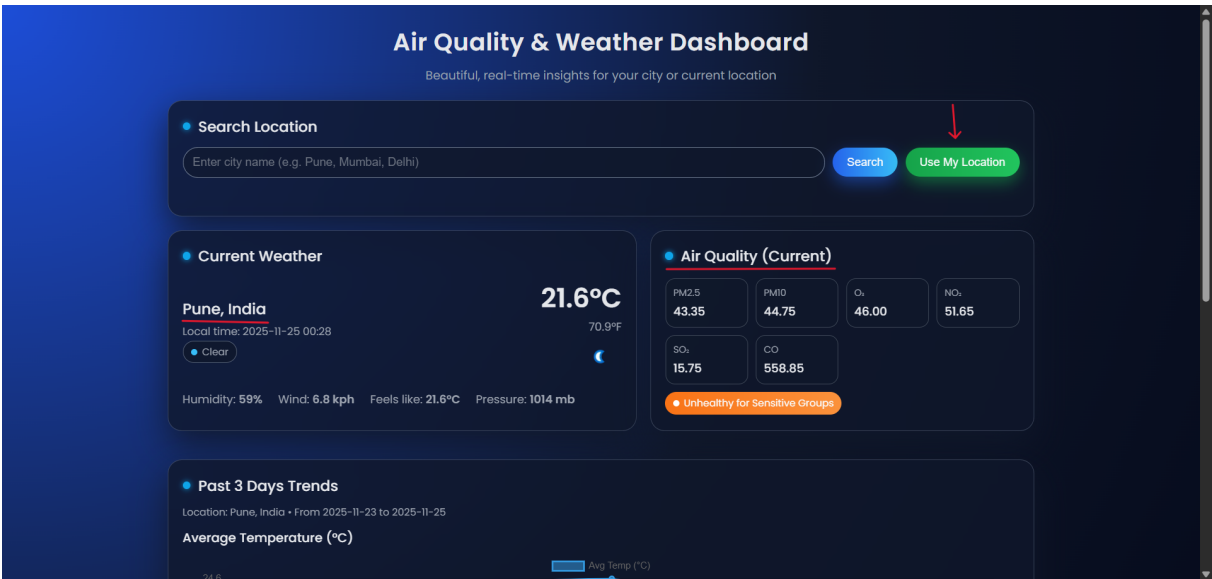


Figure 8.2: Air Quality Dashboard

8.3 Charts

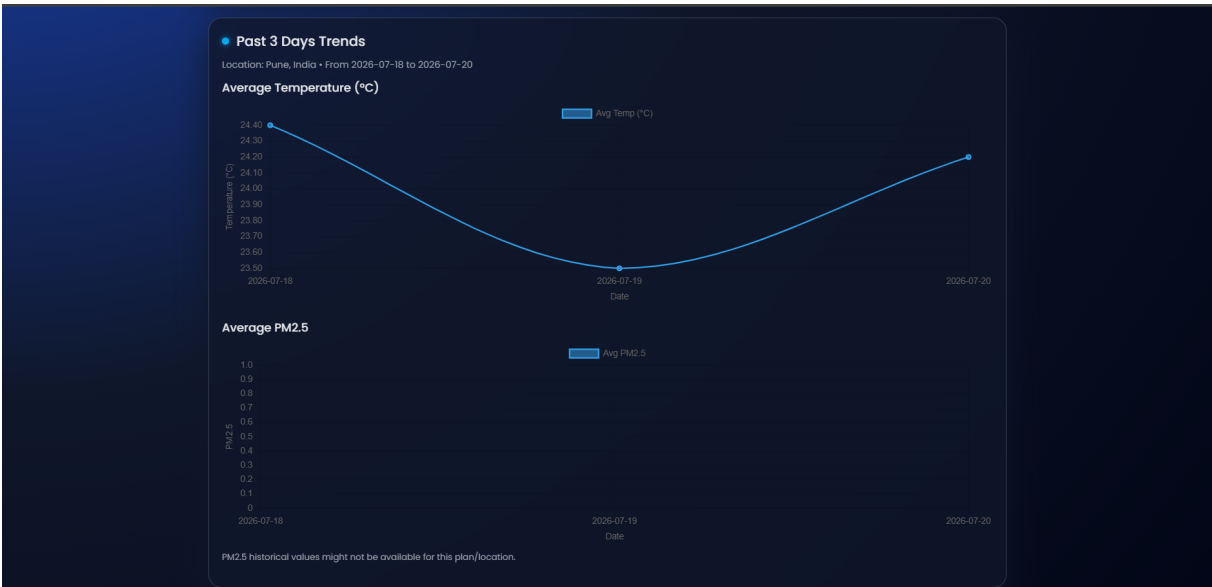


Figure 8.3: Air Quality Charts

9 Future Scope

The proposed Air Quality Monitoring Dashboard provides a strong foundation for future enhancements.

The following improvements can be incorporated in future versions.

1. Integration with IoT-based Air Quality Sensors.
2. Historical Data Storage.
3. Machine Learning based AQI Prediction.
4. AI-powered Health Recommendation System.
5. Interactive Pollution Heat Maps.
6. Smart City Integration.
7. Android and iOS Mobile Applications.
8. Email and SMS Alerts.
9. Push Notifications.
10. Cloud Deployment.
11. Multiple API Support.
12. User Login and Personalized Dashboard.
13. Pollution Trend Analysis.
14. Weather Forecast Integration.
15. Voice Assistant Support.

10 Conclusion

The Teach for Air Quality Monitoring project successfully demonstrates the development of a modern web-based application capable of providing real-time, location-aware environmental information.

The system integrates browser geolocation services with a Flask backend and the World Air Quality Index (WAQI) API to retrieve accurate air-quality parameters including AQI, PM_{2.5}, PM₁₀, Carbon Dioxide (CO₂), temperature, and humidity.

Compared with traditional static webpages, the proposed dashboard offers a dynamic, responsive, and user-friendly solution that enables users to monitor environmental conditions based on their current location.

The project also provides valuable experience in web development, API integration, JSON processing, and client-server communication using Python Flask.

Furthermore, the modular architecture allows future enhancements including IoT integration, historical data storage, predictive analytics, mobile applications, and smart-city deployment.

Overall, the developed system provides an effective platform for increasing environmental awareness while demonstrating practical implementation of modern web technologies for solving real-world problems.

Bibliography

- [1] World Air Quality Index Project.
<https://waqi.info/>
- [2] Flask Documentation.
<https://flask.palletsprojects.com/>
- [3] Python Software Foundation.
<https://docs.python.org/3/>
- [4] GitHub Documentation.
<https://docs.github.com/>
- [5] Mozilla Developer Network (MDN).
<https://developer.mozilla.org/>
- [6] WeatherAPI Documentation.
<https://www.weatherapi.com/docs/>
- [7] YouTube Tutorial: Air Quality Monitoring Project.
<https://youtu.be/QGF0D7d53i4>
- [8] YouTube Tutorial: Flask Air Quality Dashboard.
<https://youtu.be/rS5m5gtJu6M>